COLLEGE / VLG PROJECT 2025-26

Trendora

A multi-authority online shopping website
inspired by Meesho, Myntra and Flipkart

Project report - PDF copy for submission

Department of Computer Science / Information Technology

Contents

1. Certificate
2. Declaration
3. Acknowledgement
4. Abstract
5. 1. Introduction
6. 2. Literature survey
7. 3. System analysis
8. 4. System design
9. 5. Implementation
10. 6. Testing
11. 7. Results and conclusion
12. 8. Future scope
13. 9. References
14. Appendix A - How to run
15. Appendix B - User manuals by role

Certificate

This is to certify that the project titled "Trendora - A Multi-Authority Online Shopping Website" is a bona fide record of work carried out as part of the academic curriculum. The software implements a customer storefront inspired by Myntra, Meesho and Flipkart together with Seller, Admin and Owner consoles. The candidate has presented the working system, test cases and this report for evaluation.

Declaration

I declare that this project is my original work. Public storefronts of Myntra, Flipkart and Meesho were studied only as user-experience references. Brand name, source code, catalogue copy and documents are original. No live payment gateway is connected. Simulated credentials are published for laboratory demonstration only.

Acknowledgement

I thank my project guide, the Department of Computer Science / Information Technology, laboratory staff and classmates who reviewed the four-role login flow. I also thank open documentation of React, Vite and React Router.

Abstract

Indian e-commerce is defined by three consumer habits: fashion discovery (Myntra), price and deals (Flipkart) and supplier-led catalogue (Meesho). Trendora unifies those habits in one React single-page application and adds four login authorities - Customer, Seller, Admin, Owner - each with a separate home and permission set.

Customers browse eight categories, search and filter, read size charts, check PIN-code serviceability, compare up to three SKUs, heart a wishlist, apply coupons, gift-wrap, check out (UPI / card / COD simulated), track, cancel and return orders, earn Insider points, and open help tickets. Sellers list SKUs and see commissioned earnings. Admins run catalogue, order pipeline, returns, coupons and the help desk. The Owner additionally controls people, roles, store policy and GMV reports.

State lives in a Context provider persisted to localStorage (key trendora-store-v2) so a viva demo survives refresh without a paid back-end. This report documents objectives, literature, SRS, design, implementation, testing and future scope.

1. Introduction

1.1 Motivation

College students already shop on Meesho, Myntra and Flipkart. Cloning only the storefront leaves the authority model unexplained. Real platforms separate the shopper, the seller, the operations admin and the owner. Trendora therefore implements all four logins.

1.2 Problem statement

Design a responsive multi-role shopping website that covers the shopping journey of Indian fashion/general e-commerce and the back-office of a marketplace, without a live warehouse or payment gateway.

1.3 Objectives

1. Study IA of Myntra, Meesho and Flipkart.
2. Implement customer features: catalogue, search, filter, sort, PDP, size chart, PIN check, reviews, Q&A, wishlist, compare, bag, coupons, gift wrap, checkout, orders, cancel, return, addresses, Insider, studio, offers, help.
3. Implement seller studio: list SKUs, own orders, earnings after commission.
4. Implement admin desk: catalogue, order pipeline, returns, coupons, tickets, user block.
5. Implement owner console: all admin rights plus create staff, change roles, store settings, GMV reports.
6. Persist session data locally and ship PDF / Word / PPT documents.

1.4 Scope

In scope: the modules above, INR pricing, simulated payments, college documents.

Out of scope: real Razorpay capture, AWB logistics, native apps, ML recommendations.

2. Literature survey

Myntra - fashion PDP, size chart, Insider loyalty, Studio lookbooks, easy returns.

Flipkart - deal timer, coupon engine, MRP break-up, COD, order timeline.

Meesho - supplier listing, commission payout, category-first mobile bag.

Amazon Seller Central / Flipkart Seller - inventory, order states, returns desk.

Gap: a student cannot reimplement logistics. Trendora copies the nouns (bag, wishlist, COD, commission) and implements them as original React code with an honest simulation note.

3. System analysis

3.1 Actors and authority

Customer - shop, bag, pay, track, return, review, ticket.

Seller - list own products, view orders containing those SKUs, see net payout.

Admin - all catalogue, all orders, returns, coupons, tickets, block users. Cannot change owner or store policy.

Owner - superset of admin + create staff + change roles + settings (commission, free-ship, announcement) + reports.

3.2 Functional requirements (selected)

FR1 Multi-category catalogue with sellerId.

FR2 Search / filter / sort.

FR3 PDP variants, size chart, PIN.

FR4 Bag, wishlist, compare (max 3).

FR5 Coupon engine with min-cart and Insider gate.

FR6 Checkout with saved addresses, gift wrap, UPI/card/COD.

FR7 Order timeline + cancel + return.

FR8 Reviews and Q&A.

FR9 Four-role auth, block, role change.

FR10 Seller CRUD + earnings.

FR11 Admin/Owner operations desks.

FR12 Help tickets.

FR13 Notifications and Insider points.

3.3 Non-functional

Responsive, INR format, no crash on empty bag, honest security disclaimer, lab-machine friendly.

4. System design

4.1 Architecture

Browser - React pages - StoreContext (use cases) - products.js + localStorage.

Three layers: presentation, application state, persistence.

4.2 Routing

Public: /, /shop, /product/:id, /offers, /brands, /studio, /insider, /compare, /help, /login, /register.

Customer: /cart, /checkout, /orders, /returns, /addresses, /wishlist, /profile, /notifications.

Guarded: /seller/*, /admin/*, /owner/* via RequireRole.

4.3 Price algorithm

grand = max(0, subtotal - coupon + shipping + giftWrap)

shipping = 0 if subtotal >= freeShipMin else shipFee (owner configurable; default 999 / 79).

giftWrap = 49 if selected.

Insider points += floor(grand / 10).

Seller net = item sale - commission% (default 12).

4.4 Order state machine

Confirmed - Packed - Shipped - Out for delivery - Delivered - (optional) Returned.

Cancel allowed before Shipped. Return allowed only after Delivered.

4.5 Demo accounts

demo@trendora.in / demo123 (Customer)

seller@trendora.in / seller123 (Seller)

admin@trendora.in / admin123 (Admin)

owner@trendora.in / owner123 (Owner)

5. Implementation

Environment: Node.js 18+, npm, Vite 5, React 18, React Router 6, Context API, CSS design tokens.

Key modules:

src/context/StoreContext.jsx - every mutation (auth, bag, orders, catalogue, tickets).

src/data/products.js - seed catalogue, coupons, size charts, PIN helper.

src/pages/dash/* - Owner / Admin / Seller consoles.

src/components/RequireRole.jsx - route guard.

Security honesty: demo passwords sit in localStorage. Production would hash on a server (bcrypt) and issue JWT/cookies. This is stated in FAQ, slides and viva notes.

6. Testing

T1 Home hero + categories render - Pass

T2 Search "earbuds" finds PulseBuds - Pass

T3 Filter Women + sort price - Pass

T4 Two sizes of one SKU become two bag lines - Pass

T5 FESTIVE20 rejected below Rs 1999 - Pass

T6 PIN 56A rejected, 416001 accepted - Pass

T7 Customer places order; bag empties; stock drops - Pass

T8 Admin marks Packed - Shipped - Delivered - Pass

T9 Customer requests return; admin refunds - Pass

T10 Seller lists a SKU; it appears on /shop - Pass

T11 Owner creates an admin; new login works - Pass

T12 Blocked user cannot sign in - Pass

T13 Refresh keeps session (trendora-store-v2) - Pass

T14 /owner as customer redirects home - Pass

7. Results and conclusion

Trendora presents a credible Indian storefront and three staff consoles. A full purchase plus an admin status change takes under three minutes in a viva. Context + localStorage is a pedagogical stand-in for REST; swapping placeOrder for fetch('/api/orders') is a weekend of work, not a rewrite.

Conclusion: the project meets its objectives. It is demoable, documented (PDF, Word, PPT) and honest about simulation.

8. Future scope

1. Express + MongoDB API.
2. Razorpay / Stripe test mode.
3. Image upload for sellers.
4. Size recommender.
5. PWA offline catalogue.
6. Jest + React Testing Library.
7. Real SMS/email OTPs.

9. References

1. React documentation - https://react.dev
2. Vite guide - https://vitejs.dev
3. React Router - https://reactrouter.com
4. Nielsen Norman Group, e-commerce UX heuristics
5. Public storefronts of Myntra, Flipkart and Meesho (UX reference only)
6. MDN Web Docs - Web Storage API

Appendix A - How to run

cd new-shopping-website

npm install

npm run dev

Open the printed URL (default http://localhost:5173).

Production: npm run build && npm run preview.

Appendix B - User manuals by role

Customer: Login demo@trendora.in - shop - PDP - bag - FESTIVE20 - checkout - track.

Seller: Login seller@trendora.in - Catalogue - list SKU - Earnings.

Admin: Login admin@trendora.in - Orders - mark Packed / Shipped / Delivered - Returns / Tickets.

Owner: Login owner@trendora.in - People & roles - create staff - Settings - Reports.